Streams Everywhere: Comparing Local Pipelines and HTTP in Java

Daniel Briseño

Purdue University Northwest

CS 33600: Network Programming

Instructor: Roger Kraft

May 2, 2026

Abstract

This technical report compares two stream-based communication models in Java: local inter-process communication (IPC) with ProcessBuilder and network communication with HttpClient. The central claim is that both models move ordered bytes through streams, but they require different mental models because they expose control, failure, and observability in different ways. The IPC demonstration launches child Java programs, separates stdout and stderr, builds a local pipeline, redirects output to a file, and records exit codes. The HTTP demonstration sends GET requests, preserves redirect responses for inspection, prints selected headers, captures response bodies as raw bytes, and decodes text according to the charset declared in Content-Type. Runtime evidence shows that local IPC gives the parent process direct authority over stream routing and process termination, while HTTP adds protocol metadata such as status codes, Location, WWW-Authenticate, Content-Type, and Content-Length. HTTPS further adds a trust layer, shown by PKIX certificate-path failure evidence, where no HTTP response is available because TLS validation fails first. The comparison shows that streams are the common mechanism, but correctness depends on the surrounding communication system.

Keywords: Java, streams, ProcessBuilder, HttpClient, IPC, HTTP, TLS

Streams Everywhere: Comparing Local Pipelines and HTTP in Java

Introduction

Streams are one of the most important abstractions in systems programming because many programs ultimately move data as an ordered sequence of bytes. Java developers encounter this pattern when a parent process reads a child process's stdout, when two local programs are connected with a pipe, and when an HTTP client reads a response body from a remote server. These situations can look similar at first: one side produces data, another side consumes it, and the communication is handled through stream-like APIs. However, the similarity is incomplete. The surrounding control model, failure model, and debugging evidence differ depending on whether communication stays inside one operating system or crosses a network.

This report argues that Java uses streams in both local and networked communication, but developers must reason about each model differently. Local IPC with ProcessBuilder is tied to process creation, standard streams, redirection, pipelines, and exit codes. HTTP communication with HttpClient is tied to request construction, status codes, headers, response bodies, charset decoding, redirects, authentication challenges, and sometimes TLS trust validation before HTTP is even reached. The distinction matters because a stream alone does not explain whether data is correct, whether an operation succeeded, or where a failure occurred.

To examine this distinction, two focused Java demonstrations were implemented. The IPC demo launches helper programs, observes stdout and stderr, builds a pipeline, redirects output, and checks termination status. The HTTP demo fetches URLs, prints status codes and selected headers, captures response bytes, decodes text from Content-Type charset information, preserves redirects, and records 401, 403, and TLS/PKIX evidence. The report first reviews the

course and API background, then analyzes the local IPC evidence, analyzes the HTTP evidence, compares the models directly, and concludes with practical findings for Java developers.

Background

Kraft's Streams article defines a stream as an operating-system abstraction that represents a sequence of bytes and emphasizes that streams are one directional: a process either reads from an input stream or writes to an output stream (Kraft, 2026d, pp. 1-2). This definition gives the project its unifying idea. Local process output, file redirection, pipes, sockets, and HTTP bodies can all be studied as streams, even when they are embedded in different systems.

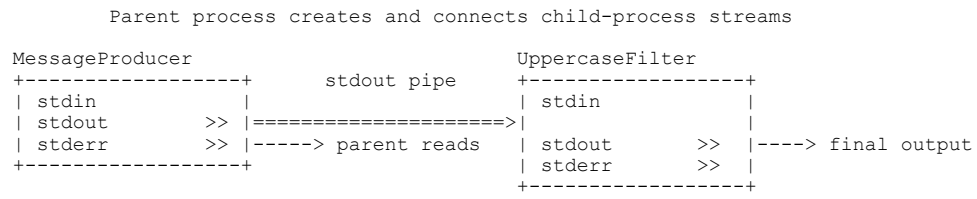
The same article explains that every process begins with three standard streams: standard input, standard output, and standard error (Kraft, 2026d, pp. 2-4). Kraft's Streams article also describes I/O redirection as the parent process telling the operating system how to connect a child process's standard streams (Kraft, 2026d, pp. 8-10). In shell syntax, this appears as operators such as <, >, 2>, and |. In Java, ProcessBuilder exposes the same concepts programmatically. Oracle's ProcessBuilder documentation describes starting operating-system processes and configuring input, output, and error redirection, including inherited I/O, file redirection, pipes, and pipelines (Oracle, n.d.-a).

Kraft's ProcessBuilder article is especially relevant because it shows that Java's default child-process behavior differs from the familiar shell model. A child process's stdout and stderr do not automatically appear in the parent's console unless the parent inherits or redirects them; by default, they are connected to pipes that the parent may read (Kraft, 2026c, pp. 3-6). The article then connects ProcessBuilder to explicit file redirection and startPipeline, which lets Java

create a multi-process pipeline similar to a shell pipeline (Kraft, 2026c, pp. 7-15). Figure 1 summarizes this local IPC stream-routing model.

Figure 1

Local IPC stream routing through a ProcessBuilder pipeline

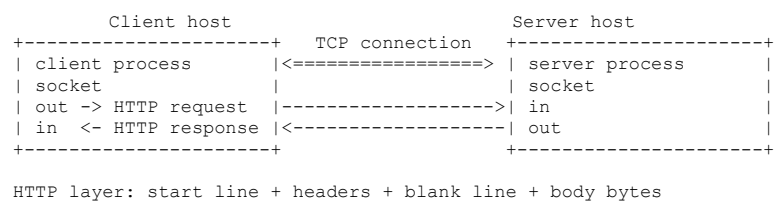


Note. Adapted from Kraft's ProcessBuilder article (Kraft, 2026c, pp. 3-7).

Networked communication extends the same stream idea across hosts. Kraft's Network Clients article explains that IPC on one machine can use operating-system mechanisms such as pipes, but IPC between machines requires network abstractions such as sockets, addresses, URLs, clients, and servers (Kraft, 2026b, pp. 3-5). The article also describes a socket connection as having input and output streams on both sides of a bidirectional network connection (Kraft, 2026b, pp. 4-5). HTTP then adds an application-layer protocol over that network connection. Kraft's HTTP article shows that an HTTP message consists of a request or status line, headers, a blank line, and an optional body (Kraft, 2026a, pp. 2-4). Figure 2 summarizes the socket-based network model that HTTP builds on.

Figure 2

Client/server socket communication across hosts



Note. Adapted from Kraft's Network clients article (Kraft, 2026b, pp. 3-5).

The Java HTTP APIs match that structure. HttpClient builds and sends requests; HttpRequest represents a configured request; HttpResponse exposes statusCode(), headers(), and body(); and BodyHandlers.ofByteArray() receives a body as raw bytes before the program decodes it (Oracle, n.d.-b, n.d.-d, n.d.-e, n.d.-f). RFC 9110 defines HTTP semantics for status codes, Content-Type, Location, and WWW-Authenticate, making those fields protocol evidence rather than arbitrary strings (Fielding et al., 2022, Sections 8.3, 10.2.2, 11.6.1, 15). Java's Charset API then provides Charset.forName(...) for resolving a declared charset before turning response bytes into text (Oracle, n.d.-g).

Local IPC in Java

The IPC demo uses three small Java files: MessageProducer, UppercaseFilter, and ProcessBuilderIpcDemo. MessageProducer writes ordinary data to stdout, writes a warning to stderr, and exits with a caller-specified code. UppercaseFilter reads stdin line by line and writes numbered uppercase output. ProcessBuilderIpcDemo wires these helpers in four ways: inherited terminal I/O, captured stdout/stderr, a producer-to-filter pipeline, and file redirection. The goal is not to build a complex application, but to expose the mechanics that matter when one Java process controls another.

The first behavior is inherited I/O. Calling inheritIO() lets the child process share the parent's terminal. This is convenient for interactive observation because output appears immediately without additional stream-reading code. It also shows a tradeoff. When stdout and stderr are inherited by the same terminal, the normal output and warning output appear in one visible stream, so the result can be convenient for a human but weaker as structured evidence.

Kraft's ProcessBuilder article explains that inheritIO() makes the child's streams behave more like the inherited stream setup used by many command-line systems (Kraft, 2026c, pp. 5-6).

The second behavior is captured stdout and stderr. Instead of sending the child's streams directly to the console, the parent reads getInputStream() and getErrorStream(), then waits for the process to terminate. The following excerpt is intentionally short; it shows the core observation point without reproducing the full source file.

Listing 1. Capturing stdout, stderr, and an exit code from a child process.

```
Process process = builder.start();
String stdout = readFully(process.getInputStream());
String stderr = readFully(process.getErrorStream());
int exitCode = process.waitFor();
```

The corresponding runtime excerpt shows why all three values matter. The child produced normal output, produced a warning on stderr, and still returned a nonzero exit code. Stream text and termination status are therefore separate forms of evidence.

Output excerpt 1. Captured stdout/stderr and child termination status.

```
-- stdout --
alpha
bravo
charlie
-- stderr --
producer: sample warning on stderr
captured exit code: 3
```

The third behavior is a pipeline. ProcessBuilder.startPipeline(...) connects the stdout of one child process to the stdin of the next child process. This matches the course model of a pipe as the connection between one process's output stream and another process's input stream (Kraft, 2026d, pp. 16-19). It also matches Kraft's ProcessBuilder article, which shows startPipeline as Java's way to construct a pipeline from a list of ProcessBuilder objects (Kraft, 2026c, pp. 15-17).

Listing 2. Creating a two-process pipeline.

```
ProcessBuilder producer = baseJavaCommand("MessageProducer", "0");
ProcessBuilder filter = baseJavaCommand("UppercaseFilter");
List<Process> pipeline = ProcessBuilder.startPipeline(List.of(producer, filter));
Process finalProcess = pipeline.get(pipeline.size() - 1);
```

Output excerpt 2. Pipeline output after the uppercase filter consumes producer stdout.

```
-- pipeline output --
01 | ALPHA
02 | BRAVO
03 | CHARLIE
pipeline final exit code: 0
```

The fourth behavior is file redirection. The demo redirects stdout to producer-output.txt while leaving stderr inherited. This split is useful because stdout and stderr are not merely different labels; they can be routed to different destinations. Normal data can be persisted while warnings remain visible on the console. In real tooling, this pattern supports logs, evidence collection, and repeatable test output without hiding urgent errors.

Listing 3. Redirecting stdout to a file while inheriting stderr.

```
builder.redirectOutput(outputFile.toFile());
builder.redirectError(ProcessBuilder.Redirect.INHERIT);
Process process = builder.start();
int exitCode = process.waitFor();
```

Taken together, the IPC results show that local process communication is direct and parent-controlled. The parent chooses the stream wiring, captures or inherits streams, creates pipelines, redirects files, and reads exit codes. This gives strong observability, but it also places responsibility on the developer to drain streams safely, avoid hidden buffering problems, and interpret stdout, stderr, and exit codes together rather than treating any one of them as the full result.

HTTP Streams in Java

The HTTP demo uses a different communication model because the other endpoint is not a child process. The Java program builds a URI-based GET request, sends it with HttpClient, and receives an HttpResponse<byte[]> object. The client uses HttpClient.Redirect.NEVER so that redirect responses remain visible instead of being followed automatically. That choice aligns with Oracle's redirect API documentation, which treats redirect handling as a configurable client policy (Oracle, n.d.-c). It also strengthens the analysis because 3xx responses can be inspected as evidence instead of being hidden by automatic client behavior.

Listing 4. Building an HTTP client that preserves redirect responses.

```
HttpClient client = HttpClient.newBuilder()
        .followRedirects(HttpClient.Redirect.NEVER)
        .build();

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .GET()
        .build();

HttpResponse<byte[]> response =
        client.send(request, HttpResponse.BodyHandlers.ofByteArray());
```

The body handler is important. By using BodyHandlers.ofByteArray(), the demo captures the response body as bytes first. The program then reads Content-Type, extracts a charset parameter if present, resolves it with Charset.forName(...), and decodes the byte array. This design reflects Kraft's HTTP article: the body is only one part of the message, and headers shape how the body should be interpreted (Kraft, 2026a, pp. 3-8). It also reflects RFC 9110, where Content-Type identifies the media type and can carry parameters such as charset (Fielding et al., 2022, Section 8.3).

Listing 5. Decoding response bytes after inspecting Content-Type.

```
Charset charset = resolveCharset(response.headers().firstValue("Content-Type"));
String body = new String(response.body(), charset);

// resolveCharset(...) falls back to UTF-8 when no supported charset is declared.
```

The successful HTTP runs show that status, headers, and charset evidence can vary even when requests return 200. GitHub's API returned application/json with charset=utf-8. Google returned text/html with charset=ISO-8859-1. Example Domain returned text/html without an explicit charset, so the program used its UTF-8 fallback. These cases demonstrate that body decoding should not be treated as an afterthought.

Output excerpt 3. Selected successful response metadata.

```
URL: https://api.github.com
Status: 200
content-type: application/json; charset=utf-8
Decoded charset: UTF-8

URL: https://www.google.com
Status: 200
content-type: text/html; charset=ISO-8859-1
Decoded charset: ISO-8859-1
```

The non-200 examples are where HTTP differs most clearly from local IPC. A redirect is not a child-process exit code; it is a valid protocol response with a 3xx status and a Location header. A 401 is not the same as a local crash; it is an authentication challenge that can include WWW-Authenticate. A 403 is a server refusal that still arrives as an HTTP response. Kraft's HTTP article groups status codes into informational, successful, redirection, client-error, and server-error ranges (Kraft, 2026a, pp. 5-6). RFC 9110 then gives standard semantics to Location and WWW-Authenticate (Fielding et al., 2022, Sections 10.2.2, 11.6.1).

Output excerpt 4. Redirect, authentication challenge, and forbidden response evidence.

```
URL: http://github.com
Status: 301
location: https://github.com/

URL: https://httpbin.org/basic-auth/user/passwd
Status: 401
www-authenticate: Basic realm="Fake Realm"

URL: https://httpbin.org/status/403
Status: 403
```

Output excerpt 5. Selected TLS/PKIX failure evidence from the debug log.

```
Fatal (CERTIFICATE_UNKNOWN): PKIX path building failed:
sun.security.provider.certpath.SunCertPathBuilderException:
unable to find valid certification path to requested target
```

The HTTP demo therefore shows that network streams are surrounded by protocol and trust metadata. The bytes in the response body matter, but the status code, headers, redirect policy, charset, and TLS outcome determine what those bytes mean and whether the client can safely read them at all.

Comparison

The strongest similarity between local IPC and HTTP is that both move ordered data through streams. In the IPC demo, MessageProducer writes bytes to stdout and UppercaseFilter consumes them from stdin. In the HTTP demo, a server writes response-body bytes and HttpClient receives them. In both cases, a reader must decide how to consume the stream, when it is complete, and how to interpret its bytes.

The strongest difference is control. In local IPC, the parent process creates the child processes and decides how their standard streams are connected. The parent can inherit streams, capture them, redirect them, pipeline them, and wait for exit codes. In HTTP, the client does not own the server process. It sends a request to a remote endpoint and receives whatever the protocol exposes. The client can configure redirect policy and body handling, but it cannot

directly inspect the server's stderr or termination status (Kraft, 2026b, pp. 3-5; Kraft, 2026c, pp. 1-7).

The error models also differ. Local IPC separates normal output, error output, and process exit status. HTTP separates status code, headers, body, and lower-layer transport or TLS failures. A nonzero exit code from a child process is not the same kind of evidence as a 401 or 403 response. The 401 response still gives protocol metadata through WWW-Authenticate, while a PKIX failure gives no HTTP response at all. This makes HTTP more expressive at the protocol level but more layered as a debugging problem (Fielding et al., 2022, Sections 11.6.1, 15; Kraft, 2026a, pp. 3-7).

Correctness also changes. For local pipelines, correctness means connecting streams properly, draining stdout and stderr, checking exit codes, and understanding the order in which output appears. For HTTP, correctness also includes URI construction, redirect handling, status-code interpretation, header parsing, charset decoding, and TLS trust validation. The two models share the stream mechanism, but they do not share the same correctness checklist. The comparison shows why the phrase "both use streams" is true but incomplete. Streams explain how bytes move. They do not explain who controls the endpoint, how failure is represented, what metadata exists, or whether another layer prevents communication before the stream can be read.

Demo Findings

The first finding is that local IPC makes stream routing visible and testable. The demo shows inherited I/O, captured stdout and stderr, a ProcessBuilder pipeline, file redirection, and exit code collection. These results reinforce the course model that standard streams can be inherited, redirected, or connected through pipes (Kraft, 2026d, pp. 8–19; Kraft, 2026c, pp. 5–15). The practical lesson is that a parent process can collect strong evidence only when it deliberately captures streams and checks exit status.

The second finding is that HTTP response handling depends on metadata, not just body bytes. Status codes, Content-Type, Location, WWW-Authenticate, and charset information determine how the response should be interpreted. A 200 response may still require charset-aware decoding, a 301 response points to a new location, a 401 response provides an authentication challenge, and a 403 response shows refusal while still returning protocol evidence.

The third finding is that HTTPS can fail before HTTP begins. The PKIX log shows that a Java request can stop during certificate-path validation before an HttpResponse object exists. That failure category has no direct equivalent in local ProcessBuilder pipelines, where the parent usually still receives an exit code and may receive stderr.

Overall, the implementation evidence supports the thesis: streams provide a shared abstraction, but the surrounding communication model determines the real engineering work. Local IPC emphasizes process wiring, stream draining, redirection, pipeline order, and exit codes. HTTP expands the checklist to URI creation, request method, redirect policy, status-code interpretation, header parsing, charset decoding, and TLS trust.

Conclusion

This project compared local IPC with ProcessBuilder and HTTP communication with HttpClient in Java. Both models are stream-oriented, but they operate in different control environments. Local IPC is parent-managed: the parent process starts child processes, wires stdout and stderr, creates pipelines, redirects files, and reads exit codes. HTTP is protocol-managed: the client receives status codes, headers, and body bytes, then interprets redirects, authentication challenges, content types, charsets, and TLS trust outcomes.

The demos show why this distinction matters. Local IPC evidence comes from stream routing and process termination, while HTTP evidence comes from protocol metadata and network-layer outcomes. A child process can return stdout, stderr, and an exit code; an HTTP request may return a 301, 401, or 403 response with meaningful headers; and an HTTPS request can fail during PKIX validation before any HTTP response exists.

The broader lesson is that developers should not treat every stream as interchangeable. Streams explain how bytes move, but they do not explain who controls the endpoint, how success or failure is reported, or what metadata determines correctness. The stream abstraction is the foundation, but the mental model must change with the communication environment.

References

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

Kraft, R. (2026a). HTTP - CS_33600. http://math.pnw.edu/~rlkraft/cs33600/for-class/readmes/Readme_12_http/

Kraft, R. (2026b). Network clients - CS_33600. http://math.pnw.edu/~rlkraft/cs33600/for-class/readmes/Readme_10_network_clients/

Kraft, R. (2026c). ProcessBuilder - CS_33600. http://math.pnw.edu/~rlkraft/cs33600/for-class/readmes/Readme_4_ProcessBuilder/

Kraft, R. (2026d). Streams - CS_33600. http://math.pnw.edu/~rlkraft/cs33600/for-class/readmes/Readme_3_streams/

Oracle. (n.d.-a). ProcessBuilder (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ProcessBuilder.html

Oracle. (n.d.-b). HttpClient (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/java/net/http/HttpClient.html

Oracle. (n.d.-c). HttpClient.Redirect (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/java/net/http/HttpClient.Redirect.html

Oracle. (n.d.-d). HttpRequest (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/java/net/http/HttpRequest.html

Oracle. (n.d.-e). HttpResponse (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/java/net/http/HttpResponse.html

Oracle. (n.d.-f). HttpResponse.BodyHandlers (Java SE 25 API specification). https://docs.oracle.com/en/java/javase/25/docs/api/java.net.http/java/net/http/HttpResponse.BodyHandlers.html

Oracle. (n.d.-g). Charset (Java SE 24 API specification). https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/nio/charset/Charset.html

Appendix A

Selected Runtime Evidence Index

The following index summarizes the selected evidence used in the report. It is included to make the runtime basis of the analysis transparent without reproducing full output files.

Evidence file	Observed result	Reason it matters
ipc-run-output.txt	Inherited I/O, captured stdout/stderr, pipeline output, file redirection, exit codes.	Shows local stream routing and process termination evidence.
producer-output.txt	alpha / bravo / charlie.	Confirms redirected stdout file contents.
http-example-output.txt	200 response with text/html and UTF-8 fallback.	Shows successful HTTP response with default decoding path.
http-github-output.txt	200 response with application/json; charset=utf-8.	Shows API body metadata and UTF-8 decoding.
http-google-output.txt	200 response with text/html; charset=ISO-8859-1.	Shows why charset-aware decoding matters.
http-redirect-output.txt	301 response with Location: https://github.com/.	Shows redirect evidence when redirects are not followed automatically.
http-401-output.txt	401 response with WWW-Authenticate: Basic realm="Fake Realm".	Shows authentication challenge metadata.
http-403-output.txt	403 response.	Shows server refusal as an HTTP response.
handshake.log	PKIX path building failed; unable to find valid certification path.	Shows TLS trust failure before HTTP response handling.